

Knowledge Is Wealth: Inferring Real-World Parameters Through Zero-Order Methods

Mattia Sabato

DAUIN, Dipartimento di Automatica e Informatica

Politecnico di Torino

Torino, Italy

mattia.sabato@studenti.polito.it

Abstract—Training agents in simulated environments provides an effective framework for deriving optimal policies that can later be deployed in real-world applications. The availability of labeled data, combined with the flexibility offered by simulation tools, enables the exploitation of information that is often inaccessible in real-world scenarios. However, this approach incurs in significant performance drops due to the *reality gap*, which arises from discrepancies between simulated and real environments caused by modeling approximations and unobservable variables. The adoption of Uniform Domain Randomization offers a promising road for mitigating this gap. Yet, this method is often characterized by inefficiencies and limited utilization of prior knowledge. In this paper, we propose an alternative approach that biases the sampling distribution towards the real-world parameters, thus addressing these limitations more effectively. Additionally, we present insights derived from analyzing the convergence behavior of the optimization algorithms and examine the influence of different parameters regulating the dynamics on generalization capabilities.

Index Terms—reinforcement learning, reality gap, domain randomization, optimization, parameters estimation

I. INTRODUCTION

One of the most significant challenges in modern Reinforcement Learning (RL) is the transfer of policies learned in simulation to real-world applications. Virtual environments, over which researchers have complete control, offer numerous advantages, such as providing accurate labels for vision tasks and precise knowledge of the underlying dynamics in control ones. However, when deploying policies trained in such controlled settings to the real world, many issues arise. Simulators often rely on assumptions that only partially align with real-world conditions, leading to compounding errors and significantly degraded performance during deployment. This phenomenon, referred to in the literature as the *reality gap*, describes the discrepancy between the environments encountered during training and those encountered at inference time, which poses substantial challenges. To mitigate it, extensive efforts have focused on reducing this discrepancy by randomizing various environment parameters to create a diverse distribution of training environments rather than relying on a single, fixed one. By exposing the agent to variations in dynamics, colors, perspectives, lighting conditions and other factors, its robustness to unseen real-world conditions at inference time is expected to improve. This approach, known as Domain Randomization (DR), shares significant conceptual similarities

with data augmentation techniques in Supervised Learning (SL), aiming at the final goal of enhancing generalization. In this paper, we analyze the benefits and challenges of using Uniform Domain Randomization (UDR) for transferring policies to new environments. Furthermore, we propose a shift in perspective to address the inefficiencies of UDR by biasing the sampling distribution toward parameters closer to those of the real world. We finally discuss the associated challenges and highlight novel insights emerging from the methods employed.

II. RELATED WORK

A. Robot Learning

The task of optimal control in robotics is typically addressed using RL methodologies. Instead of relying on models of the environment—which are often imperfect and prone to errors—robots learn optimal behavior through direct interaction with the environment. This eliminates the necessity for a perfect model but introduces further approximations due to the partial observability of the environment [1]. Another significant challenge arises from the high-dimensional, continuous state and action spaces that characterize robotic systems. Traditional value-based RL algorithms often struggle in these settings, which has led to the widespread adoption of policy-based methodologies which are more general and faster to learn [2]. A critical shared issue, however, is the need for simulated environments to train agents and derive optimal policies. Conducting real-world rollouts is often prohibitively expensive, in contraposition with the inherent data-intensive nature of RL. Virtual simulators provide a powerful alternative by offering greater flexibility and control in designing tasks and conditions. Simulated environments can generate vast amounts of data that are scarce or difficult to obtain in the real world, allowing researchers to exercise full control over what the training agent experiences. This capability is crucial for efficiently training policies while minimizing the cost and risks associated with real-world experiments.

B. Closing the reality gap

Although the use of simulators may seem to solve most of the criticalities of this framework, there will always be costs associated with such flexibility. Policies trained exclusively in simulated environments often perform poorly in real-world scenarios due to the approximations inherent in simulators

when modeling the complexities of the real world. These discrepancies lead to compounding errors, making policies trained in simulation frequently ineffective for practical applications. In [3], a discussion of the *sim2real* framework highlights both its potential and its inherent risks, concluding with a consensus on the necessity of techniques specifically designed to bridge the reality gap. Among the most promising approaches is DR, first introduced in [4], in which the agent is trained across a diverse set of environments, each characterized by randomized conditions that may include variations in dynamics, visuals, and other factors; the most naive approach implements uniform sampling distribution and it is referred as UDR. This diversity enables the resulting policy to generalize more effectively, enhancing its robustness to changes in real-world conditions by exposing it to a broader spectrum of environmental variability during training. So far, we considered DR, and in particular UDR, and discussed about how it can overcome the reality gap. However, choosing the support intervals, and eventually alternative sampling distributions, may be a subtle task with no explicit optimum. In fact, it is not easy to mathematically model the problem, and usually we are satisfied with good results of empirical evaluations. In the following, we take a different road and discuss about the possibility of estimating the real world hidden parameters. We claim that, by doing so, we can bias more effectively the sampling distribution to be close to the estimated values and make the training procedure more efficient and effective.

III. METHODOLOGY

We first introduce the mathematical formulation which will be the basis of all successive methods. We then show how UDR can improve robustness and finally propose a way to estimate real-world parameters.

A. Background

We consider a Markov Decision Process (MDP) $\mathcal{M}$, defined by a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, a reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ and a transition dynamics $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}^+$. At each time step t , an *agent* in state $s_t \in \mathcal{S}$ takes an action $a_t \in \mathcal{A}$, transitioning to a new state $s_{t+1} \in \mathcal{S}$, according to $\mathcal{P}(a_t, s_t, s_{t+1})$, and observing a reward $R_{t+1} = \mathcal{R}(s_t, a_t, s_{t+1})$. The agent's goal is to determine a parameterized policy $\pi_{\theta}^{\mathcal{M}}(a_t | s_t) \subset \Pi$ able to maximize the expected discounted cumulative reward $G_t = \sum_{t=0}^{\infty} \gamma^t R_{t+1}$ within $\mathcal{M}$. We define a sampling distribution $\mathcal{D} : (\mu, \Sigma) \in \mathbb{R}^n \times \mathbb{S}_+^n \rightarrow \mathbb{R}^n$, where $\mathbb{S}_+^n$ denotes the space of symmetric positive semi-definite matrices of size $n \times n$. This distribution generates samples ξ , which uniquely identify corresponding MDP's $\mathcal{M}_{\xi}$ by fixing parameters related to its dynamics. Consequently, a policy $\pi_{\theta}^{\mathcal{M}_{\xi_1}}$ optimized for $\mathcal{M}_{\xi_1}$ will generally differ from a policy $\pi_{\theta}^{\mathcal{M}_{\xi_2}}$ optimized for $\mathcal{M}_{\xi_2}$. We make the reasonable assumption that $\pi_{\theta}^{\mathcal{M}_{\xi_1}}$ outperforms any other $\pi_{\theta}^{\mathcal{M}_{\xi_i}}$, $i \neq 1$, on $\mathcal{M}_{\xi_1}$, i.e. $\mathbb{E}(G_t | \pi_{\theta}^{\mathcal{M}_{\xi_1}}) > \mathbb{E}(G_t | \pi_{\theta}^{\mathcal{M}_{\xi_i}})$. To evaluate the generality of a policy, we define $\phi(\pi_{\theta}^{\mathcal{M}_{\xi_i}} | \mathcal{M}_{\xi_j}, s^{(0)}) : \Pi \rightarrow \mathbb{R}$ as a stochastic measure of the *fitness* of a policy $\pi_{\theta}^{\mathcal{M}_{\xi_i}}$, trained on $\mathcal{M}_{\xi_i}$, with

initial state $s^{(0)}$, when tested on an environment modeled by $\mathcal{M}_{\xi_j}$. For simplicity, we omit the conditioning in the notation if $i = j$ or when the distinction is irrelevant, and consider $\phi(\cdot)$ to be its expected value if not otherwise specified.

Definition III.1 (Representiveness). A fitness measure $\phi(\cdot)$ is said to be *representative* if it satisfies the following conditions:

$$\|\mathbb{E}[\phi(\pi_{\theta}^{\mathcal{M}_{\xi}} | \mathcal{M}_{\xi_0})] - \mathbb{E}[\phi(\pi_{\theta}^{\mathcal{M}_{\xi_0}} | \mathcal{M}_{\xi_0})]\| \leq K \|\xi - \xi_0\| \quad (1)$$

$$\mathbb{E}[\tilde{\phi}(\pi_{\theta}^{\mathcal{M}_{\xi}})^{(t)}] - \mathbb{E}[\phi(\pi_{\theta}^{\mathcal{M}_{\xi}})] \xrightarrow[t \rightarrow +\infty]{prob.} 0 \quad (2)$$

Condition (1) requires $\phi(\cdot)$ to be K -Lipschitz continuous for some non-negative scalar constant K , ensuring that the expected fitness measure varies smoothly with changes in the parameters ξ . Condition (2) ensures that the discrepancy between the noisy realizations $\tilde{\phi}^{(t)}$ and the expected value of $\phi(\cdot)$ vanishes in probability as t increases. In the RL framework, t can be interpreted as the number of steps available to the agent in one episode. We recall that $(\pi_{\theta}^{\mathcal{M}_{\xi}})^{(t)} \xrightarrow[t \rightarrow +\infty]{dis.} \pi_{\theta}^{\mathcal{M}_{\xi}}$. When the optimal policy is not unique, the convergence in distribution applies to any single realization within the set of optimal policies. However, we conjecture that in such cases, weaker convergence guarantees may still hold.

Definition III.2 (Invariance). An MDP $\mathcal{M}_{\xi}^{\phi}$, parameterized by $\xi \sim \mathcal{D}(\cdot)$ and equipped with a measure of goodness $\phi(\cdot)$, is said to be *invariant to starting condition* if $\phi(\cdot)$ is representative and the following holds:

$$\mathbb{E}_{s^{(0)} \sim \mu(s)}[\mathbb{E}[\phi(\pi_{\theta}^{\mathcal{M}_{\xi}} | s^{(0)})^{(t)}]] \xrightarrow[t \rightarrow +\infty]{prob.} \mathbb{E}[\phi(\pi_{\theta}^{\mathcal{M}_{\xi}})] \quad (3)$$

$$\mathbb{E}_{s^{(0)} \sim \mu(s)}[(\pi_{\theta}^{\mathcal{M}_{\xi}})^{(1)} \rightarrow \dots \rightarrow (\pi_{\theta}^{\mathcal{M}_{\xi}})^{(+\infty)}] \xrightarrow[t \rightarrow +\infty]{dis.} \pi_{\theta}^{\mathcal{M}_{\xi}} \quad (4)$$

Condition (3) requires that the expected value of $\phi(\cdot)$, averaged over the initial states $s^{(0)} \sim \mu(s)$, converges to the global expected value of $\phi(\cdot)$ as the number of steps increases. Importantly, this should hold regardless of the initial state. Condition (4), for which we omit the explicit conditioning on $s^{(0)}$, imposes a stronger requirement: it demands convergence of the policy sequence $[(\pi_{\theta}^{\mathcal{M}_{\xi}})^{(i)}]_{i \in \mathbb{N}}$ to a unique optimal policy $\pi_{\theta}^{\mathcal{M}_{\xi}}$. This requirement may be difficult to satisfy in cases where the underlying graph structure of the policy sequences contains multiple sinks. Intuitively, this definition ensures that, despite starting from different initial conditions, all policies ultimately converge to a unique optimal policy and the corresponding measure of goodness $\phi(\cdot)$. This guarantees that the behavior of the policy is both reproducible and robust under varying initializations, providing reliability in performance evaluation. For example, if two policies are trained independently on the same environment, their performances on a different test environment should be similar. This can be interpreted as convergence to the same point in the space of policy graphs, ensuring consistency and invariance.

Definition III.3 (Smoothness). A family of MDP's $\mathcal{M}_{\xi}^{\phi} \subset \mathcal{M}^{\phi}$, parameterized by $\xi \sim \mathcal{D}(\cdot)$ and equipped with a measure of goodness $\phi(\cdot)$, is said to be *smooth in distortion* if

$\phi(\pi_{\theta^*}^{\mathcal{M}_{\xi_i}} | \mathcal{M}_{\xi_j})$ is representative and $\mathcal{M}_{\xi}^{\phi}$ is invariant to starting condition, provided that $\xi_i \approx \xi_j$.

Conjecture III.1. Let $\mathcal{M}_{\xi}^{\phi}$ be smooth in distortion. Then we claim the following statements are equivalent:

$$\xi \approx \xi^* \quad (5)$$

$$\mathcal{M}_{\xi}^{\phi} \approx \mathcal{M}_{\xi^*}^{\phi} \quad (6)$$

$$\pi_{\theta^*}^{\mathcal{M}_{\xi}} \approx \pi_{\theta^*}^{\mathcal{M}_{\xi^*}} \quad (7)$$

$$\phi(\pi_{\theta^*}^{\mathcal{M}_{\xi}} | \mathcal{M}_{\xi^*}) \approx \phi(\pi_{\theta^*}^{\mathcal{M}_{\xi^*}} | \mathcal{M}_{\xi^*}) \quad (8)$$

An informal proof can be found in Appendix A.

Proposition III.1. Let $\mathcal{M}_{\xi}^{\phi}$ be smooth in distortion. Consider a greedy update rule where an element of a sequence $[\xi^{(j)}]_{j \in \mathbb{N}}$ is updated toward the next iterate if and only if $\phi(\pi_{\theta^*}^{\mathcal{M}_{\xi^{(i+1)}}} | \mathcal{M}_{\xi^*}) > \phi(\pi_{\theta^*}^{\mathcal{M}_{\xi^{(i)}}} | \mathcal{M}_{\xi^*})$. Under this rule, the sequence

$$\xi^{(1)} \rightarrow \xi^{(2)} \rightarrow \dots \rightarrow \xi^* \quad (9)$$

converges to the true parameters ξ^* .

Proof. Define the neighborhood of $\xi^{(i)}$ as a ball $\mathcal{B}_r(\xi^{(i)}) = \{\xi : \|\xi^{(i)} - \xi\|_2^2 < r\}$. Using a greedy procedure, the next iterate is selected as:

$$\xi^{(i+1)} = \arg \max_{\xi \in \mathcal{B}_r(\xi^{(i)})} \phi(\pi_{\theta^*}^{\mathcal{M}_{\xi}} | \mathcal{M}_{\xi^*}) \quad (10)$$

We note that $\phi(\pi_{\theta^*}^{\mathcal{M}_{\xi^*}} | \mathcal{M}_{\xi^*})$ is bounded above by a constant, and that selecting the next iterate according to (10) ensures that the sequence $[\phi(\pi_{\theta^*}^{\mathcal{M}_{\xi^{(i)}}} | \mathcal{M}_{\xi^*})]_{i \in \mathbb{N}}$ converges to the latter. Leveraging the equivalence (8) $\iff$ (5), it follows that the parameter sequence $[\xi^{(i)}]_{i \in \mathbb{N}}$ converges to ξ^* . $\square$

B. UDR Framework

Consider an MDP $\mathcal{M}_{\xi^*}$, representing the real-world environment, parameterized by an unknown set of parameters ξ^* that governs its hidden dynamics. Our goal is to train an agent and develop a policy $\pi_{\theta^*}^{\mathcal{M}_{\xi}}$ using a simulated environment $\mathcal{M}_{\xi}$, over which we have complete control, and subsequently transfer this policy to $\mathcal{M}_{\xi^*}$. Simulation-based training offers numerous advantages, including the ability to generate virtually unlimited trajectories $[\tau(\cdot | \mathcal{P}_{\xi})^{(i)}]_{i \in \mathbb{N}}$, which can be systematically biased through ξ . However, we expect performance degradation when testing $\pi_{\theta^*}^{\mathcal{M}_{\xi}}$ on $\mathcal{M}_{\xi^*}$, as the learned policy may not fully generalize to the unseen real-world dynamics. To address this, we employ UDR. The rationale behind is analogous to data augmentation in SL. By exposing the agent to trajectories $[\tau(\cdot | \mathcal{P}_{\xi^{(i)}})^{(i)}]_{i \in \mathbb{N}}$ derived from a distribution of diverse MDP's, where $\xi^{(i)} \sim \mathcal{D}(\cdot)$, the agent learns a behavior that is general enough to be agnostic to the specific $\xi^{(i)}$ over which it was trained. In practice, UDR samples are drawn from a uniform distribution $\mathcal{S} = U(a, b)$. At the start of each episode, the parameters are fixed, resulting in an MDP that differs from both the preceding

and subsequent environment generating the training rollouts. While UDR helps in generalizing across variations in ξ , this approach can be inherently suboptimal and inefficient. Using a large interval for $U(\cdot)$ risks making the learned policy overly general – underfitting – whereas a too narrow interval may result in the policy being specialized to a limited subset of the MDP's family – overfitting. An alternative strategy involves biasing the sampling distribution $\mathcal{S}$ such that its samples are not too distant from the real parameters ξ^* . This allows for the collection of experience which is both more relevant and reflective of the real-world environment, thus improving training efficiency.

C. Zero-Order Methods

The standard UDR as presented here is quite inefficient since it does not consider any additional knowledge when considering the intervals from which to sample. We claim that, by biasing the sampling distribution to be close to the real world parameters, an agent would be able to learn faster and without considering unrealistic configurations which could hamper its performance when tested in a real-world scenario. Let's define $\mathcal{D}^*$ to be the real-world distribution generating the real-world parameters, with $\xi^* = \mathbb{E}[\mathcal{D}^*]$. We consider a distribution rather than a single configuration because we can expect the same real-world task to be characterized by very small variations when repeated. We would like to derive a sampling distribution $\mathcal{D}$ such that $\mathbb{E}[\mathcal{D}] \approx \mathbb{E}[\mathcal{D}^*]$, where $\Sigma(\mathcal{D})$ is the covariance matrix defining the amount of regularization, thus being able to perfectly mimic the real-world behavior. We assume we are able to generate a multitude of different environments modeled by $\mathcal{M}_{\xi^{(j)}}$ and parameterized by $\xi^{(j)} \sim \mathcal{D}(\mu^{(k)}, \Sigma^{(k)})$, where the index k represents the current iterate of the algorithm. For simplicity, in the following we assume to only consider the expected value of the distribution rather than samples. As discussed in the previous subsection, a greedy procedure, when selecting $\xi^{(i+1)}$, would converge to the optimum ξ^* . This would require a complete exploration of the neighborhood $\mathcal{B}_r(\xi^{(i)})$, which is in practice unfeasible if it is expensive to sample from the target environment. Instead of relying on the identification of the best candidate, and with no explicit gradient information regarding steepest ascent directions, we adopt heuristic approaches able to converge to a satisfactory solution in the least amount of real world evaluations. A general high-level description of the procedure is provided in Algorithm 1. The main component of the procedure is the `perturb` operation, which can vary in different techniques. If we could be able to completely explore the neighborhood we would always increase the fitness at each step. Since this is not possible in the *sim2real* setting, we describe here three alternatives belonging to the family of Zero-Order Methods, where no assumptions regarding derivatives are made. A more detailed analysis of the different implementations is detailed in Appendix B and [5].

1) *Naive Nelder-Mead (NNM)*: The Nelder-Mead method is an optimization algorithm particularly suited for objective functions whose derivatives are not available or difficult to

Algorithm 1: Metaprocedure for Optimal Estimation

```

 $\xi \leftarrow$  initial parameters;
 $\pi_{\theta^*} \leftarrow$  initial optimal policy trained on  $\xi$ ;
 $(\hat{\xi}^*, \text{highestFitness}) \leftarrow (\text{None}, -\infty)$ ;
while termination is False do
     $\phi(\pi_{\theta^*}^{\mathcal{M}_\xi} | \mathcal{M}_{\xi^*}) \leftarrow$  test  $\pi_{\theta^*}$  on  $\mathcal{M}_{\xi^*}$ ;
    if  $\phi(\pi_{\theta^*}^{\mathcal{M}_\xi} | \mathcal{M}_{\xi^*}) \geq \text{highestFitness}$  then
         $\hat{\xi}^* \leftarrow \xi$ ;
         $\text{highestFitness} \leftarrow \phi(\pi_{\theta^*}^{\mathcal{M}_\xi} | \mathcal{M}_{\xi^*})$ 
    end
     $\xi \leftarrow \text{perturb}(\xi)$ ;
     $\pi_{\theta_\xi^*} \leftarrow$  train optimal policy on  $\mathcal{M}_\xi$ ;
end
Return  $\hat{\xi}^*$ 

```

compute analytically. The algorithm operates on a simplex structure, leveraging geometric transformations to approximate gradient-like information and iteratively progress toward a stationary point. Although the method’s theoretical guarantees of convergence are relatively weak, they remain sufficient for a broad class of sufficiently regular objective functions.

2) *Incomplete Nelder-Mead (INM)*: A significant computational burden in the previous naive implementation is related to the evaluation of the fitness of each vertex after the shrink operation. Specifically, re-evaluating all newly generated points can be prohibitively expensive, particularly in real-world target environments. To address this, we introduce an *incomplete shrink* mechanism, where the two worst-performing points are swapped, thus altering the search direction in subsequent iterations. If this adjustment fails to improve the fitness of the worst point, only the latter is contracted toward the best one. This modification reduces computational overhead while retaining the main concepts behind the optimization process.

3) *Approximate Nelder-Mead (ANM)*: Only swapping the worst solutions can reduce the computational costs, but it risks to significantly hamper the algorithm’s performance in complex landscapes. Reflection and contraction just cause fluctuations around the best point, which cannot fully substitute for the role of the shrink operation in maintaining enough exploration. To address this, we propose an *approximate shrink* strategy. In this approach, vertices are perturbed to move closer to the best point, with their fitness values estimated using a linear approximation that leverages information from already evaluated nearby points, thus requiring only one fitness evaluation on the target environment.

IV. EXPERIMENTS

All experiments have been conducted on two distinct environments provided by the OpenAI Gym API. Validation of UDR has been performed on the HopperEnv, while the evaluation of Zero-Order methods was carried out on the CartPoleV-1 environment, referred to simply as CartPoleEnv

in the following. The selection of the CartPoleEnv has been motivated since less computationally demanding, making it a practical choice given the constraints on available resources. Additionally, its relative simplicity provides an appropriate starting point for testing and validating the proposed methodologies before applying them to more complex environments.

A. Environments

1) *HopperEnv and SAC*: The HopperEnv is defined by a state space $\mathcal{S}^H \in \mathbb{R}^{11}$ which includes the hopper’s height (1), joint angles (4), top velocities (3), and angular velocities of the hinges (3). The action space $\mathcal{A}^H \in \mathbb{R}^3$ corresponds to the torques applied to the joints. In this environment, we employed the Soft Actor-Critic (SAC) algorithm [6], aiming to maximize, during the experiments, the total cumulative reward simultaneously on the `source` and the `target` environment, where one of the parameters influencing the system’s dynamics of the former has been altered. In Table I we report the values of the dynamics parameters used, indicating which were randomized through UDR and which were kept constant. Each model was trained for 500,000 timesteps and subsequently evaluated over 50 episodes.

Coefficient	Parameter	Values[kg]	
		source	target
-	torso_mass ^{⚙️}	2.5343	3.5343
α_1	thigh_mass [🔥]	3.9270	3.9270
α_2	leg_mass [🔥]	2.7143	2.7143
α_3	foot_mass [🔥]	5.0894	5.0894

TABLE I
VALUES OF THE FIXED (⚙️) AND RANDOMIZABLE (🔥) PARAMETERS IN THE TWO DOMAINS

2) *CartPoleEnv and PPO*: The CartPoleEnv is defined by a state space $\mathcal{S}^{CP} \in \mathbb{R}^4$, which includes the cart’s position and velocity, as well as the pole’s angle and angular velocity. The action space $\mathcal{A}^{CP} \in \{0, 1\}$ represents the binary decision of moving the cart either left or right. For this environment, we used the Proximal Policy Optimization (PPO) algorithm [7], which, due to its computational efficiency, enabled the evaluation of multiple configurations in significantly less time compared to previous methods. Our focus in this environment was on optimizing the physical parameters representing the cart and pole masses, with true values of 1.0 and 0.1, respectively, assumed to be initially unknown. All models have been trained for 80,000 timesteps and tested for 10 episodes, for a total of 5 distinct runs. Notably, this limit on the amount of experience available has been chosen to prevent the agent from converging to an optimal policy. Given that this is a relatively simple environment, we aimed to encourage the agents to learn a quasi-optimal behavior that would suffice only when $\xi \approx \xi^*$, but that would likely be inadequate when this condition is not met.

B. Impact of the Reality Gap

Prior to start the experiments, it was crucial to identify optimal hyperparameters for the SAC algorithm. We introduce the notation $\langle \text{train} \rangle \rightarrow \langle \text{test} \rangle$ to highlight on which environment a model has been trained and on which it has been tested. We conducted an initial grid-search to find the best hyperparameters for the SAC, considering a $\text{source} \rightarrow \text{source}$ validation procedure and fixing the buffer size to 300,000. The values found have then been fixed for all subsequent experiments. The results of the grid search, along with the best-performing combination, are presented in Table II. After identifying the optimal hyperparameters, we performed an additional grid-search to find the optimal interval boundaries for the uniform distribution used for randomization. For each randomizable component, denoted as $\xi_{i,\text{source}}$, we defined a uniform distribution $U_i(\xi_{i,\text{source}}(1 - \alpha_i), \xi_{i,\text{source}}(1 + \alpha_i))$, $\alpha_i \in \{0.2, 0.5, 0.8\}$, with $\mathbb{E}(U_i) = \xi_{i,\text{source}}$. The results of the empirical evaluations performed under the best configurations are presented in the next section.

Hyperparameter	Search Space
batch_size	{256, 512 }
gamma	{0.99, 0.995 , 0.999}
learning_rate	{0.0003, 0.0006 }
$\langle \text{train} \rangle \rightarrow \langle \text{test} \rangle$	Reward (50 Evaluations)
source $\rightarrow$ source	1515.88 $\pm$ 8.86
source $\rightarrow$ target	1471.37 $\pm$ 133.12
target $\rightarrow$ target	1548.06 $\pm$ 7.51

TABLE II
HYPERPARAMETERS SEARCH SPACES AND PERFORMANCES WHEN TRAINED AND TESTED ON DIFFERENT DOMAINS

C. Zero-Order Methods Setting

To evaluate the effectiveness of the proposed approaches, we have conducted a series of experiments using the default hyperparameters of PPO. The rationale behind was to avoid positively biasing the performance results by tuning the values. Fixing the optimal ones could lead to overly optimistic evaluations of the agent performance in the target environment. By avoiding the incorporation of this prior knowledge, the trained agents are required to rely only on the similarities between the parameters encountered during training and those encountered during testing in order to achieve good performances. All the experiments have been run considering 30 iterations of the algorithms, initializing the points through a Latin Hypercube Sampling in the domain $[0, 2]^2$. It should be noted that the true values are contained in this range, which is a reasonable assumption in a real-world setting. Nevertheless, the width is big enough to not introduce any prior bias; we highlight that restricting the initialization to most likely parameters configurations results in more stable and accurate convergence results. The fitness evaluations necessary to guide the algorithm are obtained by first training the agent in a simulated environment parameterized by ξ , followed by testing

on the target environment parameterized by ξ^* . Since the target environment is unknown, the only feedback available regarding the quality of the solution comes from interaction with it, which is assumed to be moderately expensive and thus to be limited. The chosen fitness measure is computed as:

$$\phi(\pi_{\theta_{\xi}^*}^{\mathcal{M}_{\xi}} | \mathcal{M}_{\xi^*}) = \mathbb{E}[G_0^{\tau}] - \sqrt{\mathbb{V}[G_0^{\tau}]}, \quad \tau \sim \pi_{\theta_{\xi}^*}^{\mathcal{M}_{\xi}} \quad (11)$$

which provides a *pessimistic* estimate of the agent’s fitness in the target environment. For NNM we have tested different constants related to the different algorithm’s operations, increasing (or decreasing) all parameters simultaneously rather than using a grid-search. The values tested are: $\rho \in \{0.75, 1.00, 1.25\}$, $\chi \in \{1.50, 2.00, 2.50\}$, and $\gamma, \sigma \in \{0.25, 0.50, 0.75\}$. By varying these constants, we are able to change the step size considered at each iteration. A too large value could make the representativeness of the fitness estimate less reliable by introducing excessive distance between parameter points, whereas a too limited one could result in slow convergence. For ANM we have tested two different strategies to weight the linear combination of the fitnesses: one uses fixed weights (ANM(w)), whereas the other leverages the distances (ANM(d)). More details are provided in Appendix B.

V. RESULTS

A. Generalization Gains through UDR

When considering the $\text{source} \rightarrow \text{target}$ setting we observe lower and uncertain performances with respect to $\text{source} \rightarrow \text{source}$, as shown in Table II. This is expected because in the latter case the environment seen by the agent at training time corresponds to the one seen at test time; differently, the former suffers from a distribution shift. However, training directly on the target would be unfeasible for real-world tasks since expected to be too expensive or dangerous. On the other hand, when adopting UDR, performances increase even though the parameter being shifted has not been considered in the randomization. This allows to alleviate the performance drop due to poor generalization, at the expense of the necessity of finding good intervals to sample from.

α_1	α_2	α_3	Reward (50 Evaluations)	
			source $\rightarrow$ source	source $\rightarrow$ target
0.2	0.5	0.8	1578.62 $\pm$ 1.90	1665.17 $\pm$ 2.47
0.2	0.8	0.5	1519.60 $\pm$ 2.11	1595.00 $\pm$ 2.86
0.5	0.2	0.8	1426.92 $\pm$ 9.44	1493.94 $\pm$ 4.39
0.5	0.5	0.2	1489.90 $\pm$ 5.34	991.46 $\pm$ 27.61

TABLE III
DIFFERENT VALUES OF THE COEFFICIENTS AND REWARDS WHEN TESTING ON THE SOURCE AND TARGET ENVIRONMENTS

This insight shows how the agent has become more robust to different domains even though the major source of shift has not been regularized. This remarkable result shows how also randomizing a subset of the parameters is enough to be robust to out-of-randomization ones. Table III presents some values

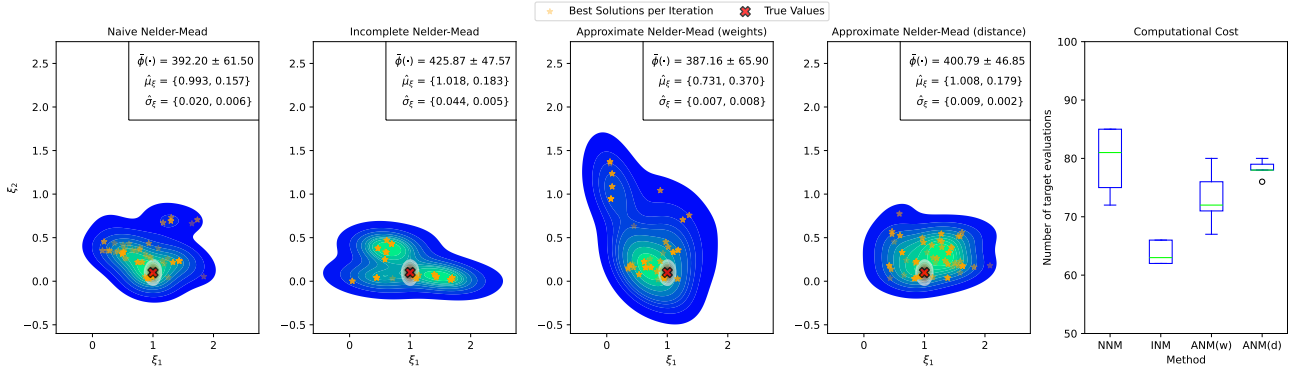


Fig. 1. Density estimates computed considering all vertexes, all iterations and all runs. All the methods are in expectation able to let the solutions get closer to the true values, with different degrees of accuracy, and stagnate around the solution. The orange stars represent the vertexes with highest fitness at each iteration of every run. The average fitness is computed across all the iterations considering all the runs, the estimate of the mean values of the parameters is computed considering the last 3 iterations of all the runs, the estimate of the standard deviation is computed considering the last 15 iterations of all the runs. On the right, the number of operations, i.e. evaluations on the target environment, for each method. All the variants present improved efficiency, with INM achieving a 20% reduction with respect to NNM, by retaining satisfactory convergence properties. We remark the fact that NNM has weak theoretical guarantees to converge towards a minimum, still the implementation is able to get close to it even though acting on a stochastic uncertain landscape.

of the coefficients used to define the sampling intervals, along with the performance of the agents trained through UDR. We highlight the fact that the performance on the test environment are usually better than the one on the training.

B. Zero-Order Methods as Parameters Estimators

The standard values of the NNM constants have shown to yield the best performance: as anticipated in the previous section, too large or too small steps lead to an inefficient exploration of the parameter space. These values have been fixed for all the other methods as well. As shown in Figure 1, all variants are able in expectation to converge close to the true values within the maximum number of iterations imposed. To improve the estimate, it is recommended to eventually restart the algorithm once reached convergence by sampling points around the best solution. Furthermore, our experiments show that the optimization process often becomes trapped in regions of the parameter space that deviate from the true parameter values. Interestingly, policies trained in these regions still exhibit strong performance when tested in the target domain. This counterintuitive outcome, which seems in contrast with the theoretical results presented in the previous sections, may be explained by the limited expressiveness of the fitness measure used. For instance, the reward function we employed primarily accounts for the agent’s survival, not considering the granularity of individual actions or the entropy of the policy. We claim that designing more expressive fitness functions tailored to the task would improve convergence and generalization outcomes. Finally, we observe how the sequences of solutions found by the algorithm are characterized by a very low variance along the pole mass axis. From empirical analyses, we further noticed that the methods consistently converge faster to range of values closer to ξ_2^* rather than ξ_1^* . We claim the algorithm generally prioritizes the pole’s mass as its optimal control, and the understanding of its dynamics, are the most accountable for the agent’s cumulative reward. On

the other hand, an accurate estimation of the cart’s mass is less relevant, as its impact on performance is indirect. Interestingly, the algorithm often converges to an overestimate of its value. This behavior likely arises because training with a heavier cart, which slows the system’s response, makes the agent robust to less challenging environments.

VI. DISCUSSION

In the *sim2real* context, the core challenge is related to ensuring generalization across different domains. While UDR mitigates performance degradations, it introduces issues related to efficiency. Specifically, in our experiments, we centered the sampling distribution around the true parameter values. However, in real-world scenarios, this approach may require substantial prior knowledge, making it impractical. Biasing the sampling distribution using Zero-Order methods offers a powerful alternative, but careful analysis of the convergence behavior of these algorithms must be taken into account (we note that algorithms using NNM as a backbone have weak guarantees to reach a minimum, but better alternatives exist; this implies that the results presented in this work represent just a lower bound). Convergence to regions far from the true parameters indicate that accurate estimates are not always necessary to derive effective policies, e.g. relative parameter ratios might be enough to produce equivalent agents with invariant performance across environments. Relevantly, the algorithms converge more efficiently and faster to a subset of parameters that predominantly influence the agent’s performance. These findings highlight the potential benefits of training agents in harder parameter configurations, focusing on the subset with the most significant impact on performance. Finally, the definition of a representative fitness function remains an open question. While our effort provides some preliminary insights using a basic formulation, future work is needed to explore more sophisticated metrics that better capture differences in policy performances across domains.

REFERENCES

- [1] J. Kober, J. A. Bagnell and J. Peters, ‘Reinforcement Learning in Robotics: A Survey,’ The International Journal of Robotics Research, Volume 32, Issue 11, pp. 1238-1274, August 2013.
- [2] P. Kormushev, S. Calinon and D. G. Caldwell, ‘Reinforcement Learning in Robotics: Applications and Real-World Challenges,’ Robotics, Volume 2, Issue 3, pp. 122-148, July 2013.
- [3] S. Höfer, K. E. Bekris, A. Handa, J. C. G. Higuera, F. Golemo, M. Mozifian, C. G. Atkeson, D. Fox, K. Goldberg, J. Leonard, C. K. Liu, J. Peters, S. Song, P. Welinder and M. White, ‘Perspectives on Sim2Real Transfer for Robotics: A Summary of the R:SS 2020 Workshop,’ CoRR, 2020.
- [4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba and P. Abbeel, ‘Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,’ IROS, 2017.
- [5] J. A. Nelder and R. Mead, ‘A Simplex Method for Function Minimization,’ The Computer Journal, Volume 7, Issue 4, pp. 308–313, January 1965.
- [6] T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, ‘Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,’ CoRR, 2018.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, ‘Proximal Policy Optimization Algorithms,’ CoRR, 2017.

APPENDIX A

INFORMAL PROOF FOR EQUIVALENCE

We provide here several intuitions motivating the reasoning behind Conjecture III.1.

Proof. We begin by demonstrating $(5) \iff (6)$. Suppose $\mathcal{M}_\xi^\phi$ is significantly different from $\mathcal{M}_{\xi^*}^\phi$, which implies $\mathcal{P}_\xi^a \not\approx \mathcal{P}_{\xi^*}^a$. Consider the norm $\|\cdot\|_{\mathcal{M}}^{[a_t]_{t \in \mathbb{N}}}$ as a measure of the divergence between the likelihoods of observing a trajectory under the same sequence of actions in the two environments. Consider the likelihood of a trajectory $\tau(s_0, s_T | \mathcal{P}_\xi) = \{s_0, s_1, \dots, s_T\}$, which is given by

$$\mathcal{L}(\tau(s_0, s_T | \mathcal{P}_\xi)) = \mu(s_0) \prod_{i=1}^T \mathcal{P}_\xi^{(a^{(i)}, s_{t-1}, s_t)} \quad (12)$$

The likelihood decreases rapidly as the trajectory length increases, whereas $\|\mathcal{M}_\xi - \mathcal{M}_{\xi^*}\|_{\mathcal{M}}^{[a_t]_{t \in \mathbb{N}_+}}$ increases as the distance between ξ and ξ^* grows. By induction, if the likelihoods differ up to a finite number of steps, their divergence will grow as the trajectory length increases. Since ξ and ξ^* influence the dynamics (and thus the transition probabilities) of the MDP’s, the only way to ensure $\mathcal{L}(\tau(s_0, s_T | \mathcal{P}_\xi)) \approx \mathcal{L}(\tau(s_0, s_T | \mathcal{P}_{\xi^*}))$, and consequently $\mathcal{P}_\xi^a \approx \mathcal{P}_{\xi^*}^a$, considering all the available actions, is to require $\xi \approx \xi^*$. The same reasoning can be applied to prove that $(6) \rightarrow (5)$, where if two trajectories are similar, must share the same transition dynamics, and thus also the same underlying parameters.

Next, we show $(6) \iff (7)$. By definition of invariance to starting condition, an agent trained on $\mathcal{M}_\xi$ will converge to the optimal policy $\pi_{\theta^*}^{\mathcal{M}_\xi}$, having adapted to the dynamics of $\mathcal{M}_\xi$.

Similarly, an agent trained on $\mathcal{M}_{\xi^*}$ will converge to $\pi_{\theta^*}^{\mathcal{M}_{\xi^*}}$. By assumption, both policies maximize $\mathbb{E}[G_t | s_t]$, given the respective environment dynamics. If these dynamics are too dissimilar, the policies will myopically diverge in the short term. However, if the dynamics are sufficiently similar, the

optimal policies must also exhibit similarity having learned the reactive optimal behavior with respect to the current state. Also the viceversa is straightforward to see, so that if the policies are similar when tested on the same environment, it means that the intrinsic reward process presents similarities. Being the reward function deterministic by assumption, and thus influenced only by the transition dynamics and the probabilities of the stochastic policies, the only way to adapt to the transition dynamics is to adapt the policy. However, if the policies are similar for any state, it means that the expected trajectories conditioned on the policy starting from any state should be similar as well. Being the likelihood (9) dependent both on the action taken and the transition dynamics, if the action is approximately fixed the only way to make the likelihood similar is to require similar underlying dynamics, and thus similar underlying MDP’s.

We now prove $(7) \rightarrow (8)$. By definition of representativeness, if two policies $\pi_{\theta^*}^{\mathcal{M}_\xi}$ and $\pi_{\theta^*}^{\mathcal{M}_{\xi^*}}$ are similar, their corresponding fitness measures when tested on $\mathcal{M}_{\xi^*}$ must also be similar.

Finally, we demonstrate $(8) \rightarrow (6) \rightarrow (5)$. If two policies are trained on different environments but they exhibit similar fitness when tested on the same, this means that the policies have been trained on similar environments. If they were not, policy $\pi_{\theta^*}^{\mathcal{M}_\xi}$, when tested on $\mathcal{M}_{\xi^*}$, would try to maximize its return, governed by different dynamics, thus leading to a likely smaller fitness; recall the assumption that a policy trained and tested on the same environment is expected to outperform any other policy trained on a different environment. Having similar underlying MDP’s, then also the underlying parameters must be similar. An alternative road is to make the additional reasonable assumption in (1) that the distance between the parameters is bounded and that the fitness function is actually reasonably proportional to ξ so that it has a gradient such that $\|\nabla_\xi(\phi(\pi_{\theta^*}^{\mathcal{M}_\xi} | \mathcal{M}_0))\|_2 > B, B \in \mathbb{N}_+$ reasonably large, i.e. we require the K-Lipschitz function $\phi(\cdot)$ to not be flat. Exploiting this additional condition, we can directly prove that $(8) \rightarrow (5)$. $\square$

APPENDIX B

ZERO-ORDER ALGORITHMS

We briefly present the main ideas behind the NNM algorithm and motivate the choices behind its variants.

A. Naive Nelder Mead

Let’s consider the following unconstrained continuous maximization problem:

$$\max_{x \in \mathbb{R}^n} f(x), \quad f: \mathbb{R}^n \rightarrow \mathbb{R} \quad (13)$$

which we aim to solve numerically through an iterative optimization algorithm. Let $\mathcal{S}_{n+1}^{(k)}$ be a non-degenerate simplex in $\mathbb{R}^n$ at iteration k , whose set of vertexes $\mathcal{X}^{(k)} = \{x_1^{(k)}, \dots, x_{n+1}^{(k)}\}$ contains the candidate solutions for (13). We reorder the points in $\mathcal{X}^{(k)}$ so that $f(x_1^{(k)}) \leq f(x_2^{(k)}) \leq \dots \leq f(x_{n+1}^{(k)})$. Then, we apply the iterative optimization

algorithm repeatedly until we reach a termination criterion. In each iteration we can perform a variable number of operations trying to improve $f(x_1^{(k)})$ until we reach convergence to a stationary point. In our setting we are particularly interested to the number of function evaluations, since each is assumed to be expensive in the *sim2real* scenario.

Operation	#evaluations
Reflection	1
Expansion	1
Contraction	1
Shrink	n

TABLE IV
COST OF EACH OPERATION IN THE NELDER-MEAD METHOD

It is obvious, as shown in Table IV, that the shrink operation could be extremely expensive, thus we explore alternatives to reduce this cost by keeping good convergence behavior. To partially alleviate this issue, and avoid stagnation, both in this method, and all the variants considered, if the worst solution does not improve after 2 consecutive shrinks, the simplex is reinitialized. This leads to $6 + 2n + (n + 1)$ evaluations across 3 iterations in a worst case scenario, i.e. $\frac{13}{3} = 4.33$ evaluations per iterate.

B. Incomplete Nelder Mead

We aim to reduce the number of evaluations by observing the following. The reflection operation explores along the direction

$$p_1^{(k)} = \underbrace{x_{barycenter}^{(k)}}_{\frac{1}{n+1} \sum_{i=1}^{n+1} x_i} - x_1^{(k)} \quad (14)$$

centered in $x_{barycenter}^{(k)}$, and moves in the space proportionally to a constant ρ . We claim that if moving along $p_1^{(k)}$ does not improve over $f(x_1^{(k)})$, it is worth exploring $p_2^{(k)}$, where we substitute $x_1^{(k)}$ with $x_2^{(k)}$, before computing an expensive shrink considering all the points. If this swap again does not allow to improve over the worst point, we perform a shrinking considering only the worst point. In a best case scenario the number of evaluations per shrink reduces to 0, whereas in a worst case scenario it increases to $6 + 1 + (n + 1)$ across 3 iterations, i.e. $\frac{10}{3} = 3.33$ evaluations per iterate.

C. Approximate Nelder Mead

Since we do not want to completely avoid the shrink operation at each iteration, we try to find a way to mimic it without the need of evaluating the solutions. During the shrinking, each point $x_i^{(k)}, i < n + 1$, moves along the direction $p_i^{(k)} = x_{n+1}^{(k)} - x_i^{(k)}$, centered in $x_{n+1}^{(k)}$. We aim to compute a best-guess for each shrunked point through a linear combination of its current value $f(x_i^{(k)})$, the estimated value $\hat{f}(x_1^{(k)} + \sigma(x_i^{(k)} - x_1^{(k)}))$ and the value of the barycenter $f(x_{barycenter}^{(k)})$. Thus, we have the following:

$$\tilde{x}_i^{(k+1)} = \frac{\overbrace{x_i^{(k)}}^{s_1}}{3} + \frac{\overbrace{x_1^{(k)} + \sigma(x_i^{(k)} - x_1^{(k)})}^{s_2}}{3} \quad (15)$$

$$+ \frac{\overbrace{x_{barycenter}^{(k)}}^{s_3}}{3}$$

$$\tilde{f}(\tilde{x}_i^{(k+1)}) = \langle w, F(s_1, s_2, s_3) \rangle \quad (16)$$

where $w \in \mathbb{R}^3$ is a vector of weights and $\langle \cdot, \cdot \rangle$ is the usual scalar product. Regarding $F : \mathbb{R}^{3 \times n} \rightarrow \mathbb{R}^3$, we consider it to be a vector valued function evaluating $f(\cdot)$ in multiple points. However, it does not actually compute $f(s_2)$ in an exact way, but rather it estimates it as:

$$\hat{f}(s_2) = (1 - \sigma)f(x_i^{(k)}) + \sigma f(x_1^{(k)}) \quad (17)$$

obtaining:

$$\langle w, F(s_1, s_2, s_3) \rangle = w_1 \overbrace{f(s_1)}^{no\ eval} + w_2 \overbrace{\hat{f}(s_2)}^{no\ eval} + w_3 \overbrace{f(s_3)}^{eval} \quad (18)$$

Finally, we replace $x_i^{(k+1)}$ with $\tilde{x}_i^{(k+1)}$ and $f(x_i^{(k+1)})$ with $\tilde{f}(\tilde{x}_i^{(k+1)})$ at the next iteration. In our experiments, we considered two scenarios for fixing w : one in which the values are fixed and equal to $w = [0.2, 0.2, 0.6]$, and another where the values are inversely proportional to the euclidean distance between $\tilde{x}_i^{(k+1)}$ and the components of (s_1, s_2, s_3) . In a best case scenario the number of evaluations reduces to 1, whereas in a worst case scenario it increases to $6 + 3 + (n + 1)$ across 3 iterations, i.e. $\frac{12}{3} = 4$ evaluations per iterate.